1) For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.

```
mysql> USE DBMS_CO3;
Database changed
mysql> CREATE TABLE STUDENT_COURSE (
    ->     SID INT,
    ->     SName VARCHAR(50),
    ->     CID INT,
    ->     CName VARCHAR(50),
    ->     Instructor VARCHAR(50),
    ->     Grade CHAR(2),
    ->     PRIMARY KEY (SID, CID)
    -> );
Query OK, 0 rows affected (0.21 sec)

mysql>
mysql> INSERT INTO STUDENT_COURSE VALUES
    -> (101,'John',201,'DBMS','Arun','A'),
    -> (101,'John',202,'OS','Kumar','B'),
    -> (102,'David',201,'DBMS','Arun','A');
Query OK, 3 rows affected (0.02 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM STUDENT_COURSE;
+-----+-------+-----+-------+------------+-------+
| SID | SName | CID | CName | Instructor | Grade |
+-----+-------+-----+-------+------------+-------+
| 101 | John  | 201 | DBMS  | Arun       | A     |
| 101 | John  | 202 | OS    | Kumar      | B     |
| 102 | David | 201 | DBMS  | Arun       | A     |
+-----+-------+-----+-------+------------+-------+
3 rows in set (0.01 sec)
```

2) Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.

```
mysql> CREATE TABLE STUDENT_1NF (
    ->     SID INT,
    ->     Name VARCHAR(50),
    ->     Course VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.05 sec)

mysql>
mysql> INSERT INTO STUDENT_1NF VALUES
    -> (101,'John','DBMS'),
    -> (101,'John','OS'),
    -> (102,'David','CN');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM STUDENT_1NF;
+------+-------+--------+
| SID  | Name  | Course |
+------+-------+--------+
|  101 | John  | DBMS   |
|  101 | John  | OS     |
|  102 | David | CN     |
+------+-------+--------+
3 rows in set (0.00 sec)
```

3) Given relation R(A,B,C,D,E) with FDs: A→BC, BC→D, D→E. Analyze and decompose it to 2NF eliminating partial dependencies.

```
mysql> CREATE TABLE R (
    ->     A INT PRIMARY KEY,
    ->     B VARCHAR(20),
    ->     C VARCHAR(20),
    ->     D VARCHAR(20),
    ->     E VARCHAR(20)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> INSERT INTO R VALUES
    -> (1,'B1','C1','D1','E1'),
    -> (2,'B2','C2','D2','E2');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM R;
+---+------+------+------+------+
| A | B    | C    | D    | E    |
+---+------+------+------+------+
| 1 | B1   | C1   | D1   | E1   |
| 2 | B2   | C2   | D2   | E2   |
+---+------+------+------+------+
2 rows in set (0.00 sec)
```

4) Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.

```
mysql> CREATE TABLE DEPARTMENT(
    ->      DeptID INT PRIMARY KEY,
    ->      DeptName VARCHAR(50),
    ->      ManagerID INT
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> CREATE TABLE EMPLOYEE(
    ->      EmpID INT PRIMARY KEY,
    ->      EmpName VARCHAR(50),
    ->      DeptID INT,
    ->      FOREIGN KEY (DeptID)
    ->      REFERENCES DEPARTMENT(DeptID)
    -> );
Query OK, 0 rows affected (0.05 sec)

mysql>
mysql> INSERT INTO DEPARTMENT VALUES
    -> (10,'CSE',501),
    -> (20,'ECE',502);
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO EMPLOYEE VALUES
    -> (101,'John',10),
    -> (102,'David',20);
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM EMPLOYEE;
+--------+----------+---------+
| EmpID  | EmpName  | DeptID  |
+--------+----------+---------+
|    101 | John     |      10 |
|    102 | David    |      20 |
+--------+----------+---------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM DEPARTMENT;
+---------+-----------+------------+
| DeptID  | DeptName  | ManagerID  |
+---------+-----------+------------+
|      10 | CSE       |        501 |
|      20 | ECE       |        502 |
+---------+-----------+------------+
2 rows in set (0.00 sec)
```

5) Verify whether R(A,B,C,D) with FDs: AB→C, C→D, D→A is in BCNF. If not, decompose it into BCNF.

```
mysql> CREATE TABLE R3(
    ->     A INT,
    ->     B INT,
    ->     C INT,
    ->     D INT
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> INSERT INTO R3 VALUES
    -> (1,1,5,10),
    -> (2,2,6,20);
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM R3;
+------+------+------+------+
| A    | B    | C    | D    |
+------+------+------+------+
|    1 |    1 |    5 |   10 |
|    2 |    2 |    6 |   20 |
+------+------+------+------+
2 rows in set (0.00 sec)
```

6) Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.

```
mysql> CREATE TABLE R1(
    ->      A INT,
    ->      B INT
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> CREATE TABLE R2(
    ->      B INT,
    ->      C INT
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> INSERT INTO R1 VALUES
    -> (1,100),
    -> (2,200);
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO R2 VALUES
    -> (100,500),
    -> (200,600);
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT *
    -> FROM R1
    -> JOIN R2
    -> ON R1.B=R2.B;
+------+------+------+------+
| A    | B    | B    | C    |
+------+------+------+------+
|    1 |  100 |  100 |  500 |
|    2 |  200 |  200 |  600 |
+------+------+------+------+
2 rows in set (0.01 sec)
```

7) Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.

```
mysql> CREATE TABLE Teacher_Subject(
    ->      TID INT,
    ->      Subject VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> CREATE TABLE Teacher_Hobby(
    ->      TID INT,
    ->      Hobby VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> INSERT INTO Teacher_Subject VALUES
    -> (1,'DBMS'),
    -> (1,'OS');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Teacher_Hobby VALUES
    -> (1,'Reading'),
    -> (1,'Music');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM Teacher_Subject;
+------+---------+
| TID  | Subject |
+------+---------+
|    1 | DBMS    |
|    1 | OS      |
+------+---------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Teacher_Hobby;
+------+---------+
| TID  | Hobby   |
+------+---------+
|    1 | Reading |
|    1 | Music   |
+------+---------+
2 rows in set (0.00 sec)
```

8) Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).

```
mysql> CREATE TABLE Project_Supplier(
    ->     ProjectID INT,
    ->     SupplierID INT
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> CREATE TABLE Project_Part(
    ->     ProjectID INT,
    ->     PartID INT
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> CREATE TABLE Supplier_Part(
    ->     SupplierID INT,
    ->     PartID INT
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> SELECT * FROM Project_Supplier;
Empty set (0.00 sec)

mysql> SELECT * FROM Project_Part;
Empty set (0.00 sec)

mysql> SELECT * FROM Supplier_Part;
Empty set (0.00 sec)
```

9) Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.

```
mysql> CREATE TABLE Department(
    ->     DeptID INT PRIMARY KEY,
    ->     DeptName VARCHAR(50),
    ->     HOD VARCHAR(50)
    -> );
ERROR 1050 (42S01): Table 'department' already exists
mysql>
mysql> CREATE TABLE Student(
    ->     RegNo INT PRIMARY KEY,
    ->     Name VARCHAR(50),
    ->     DeptID INT,
    ->     FOREIGN KEY (DeptID)
    ->     REFERENCES Department(DeptID)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> INSERT INTO Department VALUES
    -> (1,'CSE','Dr.Raj'),
    -> (2,'ECE','Dr.Kumar');
ERROR 1366 (HY000): Incorrect integer value: 'Dr.Raj' for column 'ManagerID' at row 1
mysql>
mysql> INSERT INTO Student VALUES
    -> (1001,'John',1),
    -> (1002,'David',2);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (
`) REFERENCES `department` (`DeptID`))
mysql>
mysql> SELECT * FROM Student;
Empty set (0.00 sec)

mysql> SELECT * FROM Department;
+--------+----------+-----------+
| DeptID | DeptName | ManagerID |
+--------+----------+-----------+
|     10 | CSE      |       501 |
|     20 | ECE      |       502 |
+--------+----------+-----------+
2 rows in set (0.00 sec)
```

10) Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.

```
mysql> CREATE TABLE CandidateKey(
    ->     A INT PRIMARY KEY,
    ->     B VARCHAR(20),
    ->     C VARCHAR(20),
    ->     D VARCHAR(20),
    ->     E VARCHAR(20)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> INSERT INTO CandidateKey VALUES
    -> (1,'B1','C1','D1','E1'),
    -> (2,'B2','C2','D2','E2');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> SELECT * FROM CandidateKey;
+---+------+------+------+------+
| A | B    | C    | D    | E    |
+---+------+------+------+------+
| 1 | B1   | C1   | D1   | E1   |
| 2 | B2   | C2   | D2   | E2   |
+---+------+------+------+------+
2 rows in set (0.00 sec)
```